Scrum Studio

What the MCP and skill-set do

Scrum Studio is a local project workspace for humans and AI agents. It combines a Trello-style Kanban board with structured task specifications, ownership, dependencies, verification and resumable work sessions. The interface and task content use English.

The website is the human-facing dashboard. The MCP server exposes the same project operations to compatible AI clients and works independently of the web server. The skill-set supplies the workflow instructions that tell agents when and how to use those operations.

Core capabilities

- Projects and tasks: descriptions and acceptance criteria at creation, project-local task codes, priorities, categories, owners, checklists and multiple dependencies.
- Workflow: TODO, In Progress, Blocked, In Review, Done and Cancelled. Completion and cancellation resolve dependent links and return fully unblocked work to TODO with authored comments.
- Coordination: task claims, leases, configurable active-task limits and advisory file reservations isolated per project checkout.
- Shared context: authored updates, comments, continuity summaries, Daily notes, Retrospectives, human Review notes and project Chat.
- Visibility: active-worker indicators, time-in-state history, flow charts, local MCP overhead metrics and optional client-reported task token totals.
- Local operation: Windows and macOS launchers; per-project databases and optional portable JSON snapshots for project-data handoffs.

Intended agent workflow

1. Register or reuse the complete currently known authorized scope before starting implementation. New tasks begin in TODO with description and acceptance criteria already present.
2. Prefer small, independently verifiable outcomes. Explain cohesive grouping; avoid speculative backlog and one-task-per-file fragmentation.
3. Resume with the latest relevant retrospective, cached policy and a review of obsolete own file reservations.
4. Claim a task and planned files, do useful work between calls, and record meaningful progress or blockers.
5. Complete only with actual verification evidence. On interruption, save recorded progress and remaining checks, return unfinished work to TODO and release owned reservations.

Task creation alone does not authorize execution. The skills provide procedural guidance; the server enforces supported state, ownership and policy checks.

Practical assessment and efficiency

What improves

Organization, traceability and session continuity improve clearly. A later agent can recover the task specification, latest progress, pending checks and ownership without reconstructing the entire conversation. Claims and file reservations help prevent overlapping work across agents. These benefits matter most in longer projects, interrupted sessions and concurrent work.

What still costs time

Coordination adds calls: planning, claiming files, updating states and closing work. For a tiny single-agent change, this overhead can make the change slower. For longer work it can be offset by avoiding duplicated investigation, lost context and conflicting edits. File reservations remain advisory and do not stop an unrelated editor from changing a file.

Measured response reductions

An isolated, equal-input synthetic fixture compared compact UTF-8 JSON responses before and after the efficiency changes. It used a bounded task, a reported-token record and cached settings, without an automatic claim.

Measurement	Before	After	Reduction
Compact Light startup	1,112 bytes	678 bytes	39.0%
Compact Standard startup	1,130 bytes	696 bytes	38.4%
Default task read	752 bytes	502 bytes	33.2%
Four routine skill entrypoints	6,875 bytes	4,493 bytes	34.6%

These measurements demonstrate less coordination text. They are not tokenizer counts, a full-workload speed benchmark or proof of equivalent subscription-quota savings. Current responses may vary with task data and installed version. The fixture can be rerun with scripts/benchmark_coordination.py.

Working recommendation

Use Auto/Light for bounded single-agent work and Standard when dependencies, concurrency or scope require it. Keep descriptions and acceptance criteria concise. Load history, metrics, costs and specialist skills only when needed; reuse cached versions and combined start/finish operations. Daily, Chat and human Review are not mandatory per-task ceremonies. Preserve the brief end-of-session retrospective and genuine human follow-ups.

The next useful evaluation is several comparable real tasks, keeping model, context, provider and telemetry coverage comparable. Measure coordination calls, payload size and available task-attributable tokens. Do not add repeated model calls or idle model polling just to measure the process.

Operation, safeguards and limits

Wakeup and Stop

With an explicitly connected CLI bridge, Wakeup sends a continuation instruction to its configured Codex or Claude worker. Each Wakeup starts a new CLI conversation that recovers work from Scrum task context; it does not inject a message into an existing Desktop chat.

The Scrum Studio Chat shows the instruction, a bounded final response when available, failures and interruptions. Full tool and reasoning streams stay in local worker logs. A bridge must remain connected; opening the website alone does not start a worker.

Stop interrupts only the bridge's own worker process group. After process exit, the handoff preserves the latest recorded progress, unchecked steps, acceptance to recheck and planned files. It releases owned task/file reservations and records pause notes. Stop is not a rollback and never marks unfinished work Done. An interrupted model cannot be guaranteed to produce a new final answer; the fallback uses existing records without another model call.

Quotas and task costs

Default cooperative usage thresholds are 92% of the five-hour allowance or 94% of weekly usage, configurable per project. Unknown or stale telemetry pauses guarded work. Provider update delays and safe stopping boundaries mean these checks cannot guarantee an exact provider-level cutoff.

Account quota is shared across simultaneous agents and cannot be assigned reliably to individual tasks from account-level changes. Optional estimates are judgments, not provider measurements. Reported task tokens are stored only when a client supplies attributable input/output counts, deduplicated by agent and task-specific run. Missing data stays Not available; partial reports are not complete billing. Cached input tokens must not be counted twice.

Storage and sharing

Each registered project keeps its working databases and reservations locally under its own .scrum directory. Other project repositories may deliberately share portable project.json and snapshot.json files, while keeping .scrum/local out of Git. Locks and live sessions are never portable handoff data.

This application repository excludes its entire development .scrum board, data directory, logs and local configuration from Git. A clean clone distributes the website, backend, MCP, skills, tests and launchers without the developer's board or machine registry. Create or open a project after setup.

Compatibility and validation

The documented release baseline is MCP 2.7.0 with 34 tools and 23 skills. The website and MCP use Python 3.10+ without third-party runtime packages. AI workers require their provider CLI, account and applicable permissions; the website itself does not require an AI account. PDF generation dependencies are documentation tooling, not app runtime requirements.

Codex and Claude CLI paths are supported in the bridge. Native macOS and real Claude-provider execution were not validated on the Windows development host; automated compatibility tests do not replace those checks. No guaranteed live synchronization with this Desktop conversation is claimed.